Lösningsgenomgång

Chalmers Challenge 2022

Last Minute

We construct problems by pairing themes with techniques. Some of these can be reused, others can at most be used once.

Auniq non-reusable and Areuse reusable techniques. Buniq and Breuse for themes. WLOG Areuse <= Breuse, otherwise swap A<=>B

3 cases:

- Areuse = Breuse = 0. Only unique components, one is a bottleneck: **min(Auniq, Buniq)**
- Areuse = 0. Only Auniq for A, use same reusable B always: **Auniq**
- Both reusable available. **Auniq** + **Buniq** + **Areuse** * **Breuse**

```python
au, bu, ar, br = [int(i) for i in input().split()]

if ar == 0 and br == 0:
    print(min(au, bu))
    exit()

if ar == 0:
    print(au)
    exit()

if br == 0:
    print(bu)
    exit()

print(au + bu + (ar * br))

```

By: Mateusz Drwal

Tabs and Spaces

We have F <= 10 files indented with spaces for which we want to find the optimal tab length to minimize file size. Each file is given as a list (length <= 20) of the number of indentation spaces on each line. The maximum line length is 79 spaces, so we can check each tab length.

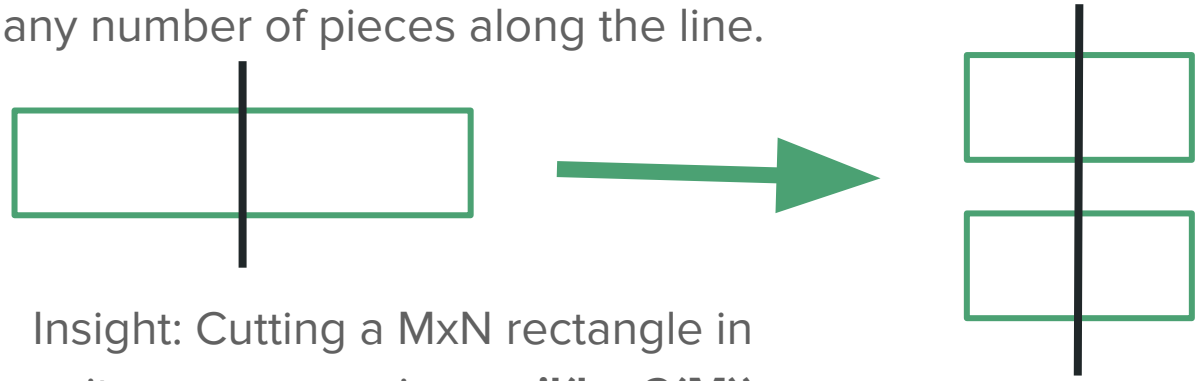
A line with N spaces with tab size T will require floor(N/T) + (N % T) bytes. We try all tab sizes and pick the best result.

```python
F =int(input())
A = list()
for filenu in range(F):
    L = int(input())
    for i in range(L):
        A.append(int(input()))


best = 1
bestVal = 0

for tab in range(1, 200):
    cnt = sum((a//tab)*(tab-1)for a in A)
    if cnt > bestVal:
        bestVal = cnt
        best = tab

print(best)
print(bestVal)

```

By: Jonathan Nilsson

Pizza Slicing

Problem: Cut a rectangular AxB pizza in at least N square slices using as few cuts as possible. We may move the pieces around between cuts, and each cut can cut any number of pieces along the line.



Insight: Cutting a MxN rectangle in unit squares requires **ceil(log2(M)) + ceil(log2(N))** cuts.

Square size? First divide out the gcd of A and B, then cut the DxD squares into K^2 squares, where K is the smallest number such that A*B*K^2/D^2 >= N

Pizza Slicing

```python
from math import *
N, A, B = map(int, input().split())
w = max(A, B)
h = min(A, B)

g = gcd(w, h)
w //= g
h //= g

def numPieces(gm):
    a = gm * h
    b = gm * w
    return a * b

gm = int(ceil(sqrt(N / (w * h))))

while numPieces(gm) < N:
    gm += 1

def cutsToMake(slices):
    return int(ceil(log2(slices)))

print(cutsToMake(gm * h) + cutsToMake(gm * w))

```

Multiplying A and B overflows 64 bit integers, so in c++ one has to be careful or use 128 bit integers.

By: Erik Amirell Eklöf

Cash Transport

N banks, M transports, K hits (all <= 3e5). Each bank starts with a balance of a_i. Transports occur in order and bring all money from bank a to bank b. You can rob at most K transports, taking all their money. How much money can you take at most?

Insight: If money is ever transferred A -> B -> C, it is always better to rob the last leg and if we do, hitting the first leg is unnecessary. Hence we can ignore all transports which lead to banks from which a transport will leave in the future.

Among these transports, we just hit the K most lucrative ones.

Cash Transport

```
1
2  N, M, K = map(int, input().split())
3
4  bal = [int(x) for x in input().split()]
5  balances = bal.copy()
6  sources = [[] for _ in range(N)]
7
8  moves = [tuple(map(int, input().split())) for _ in range(M)]
9  for a, b in moves:
10     sources[b].append(balances[a])
11     sources[a] = []
12     balances[b] += balances[a]
13     balances[a] = 0
14
15 final = []
16 for source in sources:
17     for transaction in source:
18         final.append(transaction)
19
20 print(sum(sorted(final, reverse=True)[:K]))
21
```

By: Viktor Lundkvist

Chalmers Coin

Task: To find the x < 2^36 that minimizes hash(k * 2^36 + x) for our very bad hash function hash(u64) -> u64.

10 points: Check random x for 1 second in python

30 points: Check random x for 1 second i c++

```python
def hash(x):
    # int to list of bits
    def tolist(x):
        return [(x>>i)&1 for i in range(63, -1, -1)]

    # list of bits to int
    def toint(x):
        return sum([x[63-i]<<i for i in range(0, 64)])

    assert 0 <= x < 2**64
    x = tolist(x)
    for i in range(512):
        y = 4*x[i%64] + 2*x[(i+2)%64] + x[(i+10)%64]
        x[i%64] = [1,1,0,0,1,1,0,1][y]
    x = toint(x)
    for i in range(16):
        x += toint(tolist(x)[::-1])
        x ^= 3**40
        if(x >= 2**64): x >>= 1
    assert 0 <= x < 2**64
    return x
```

Chalmers Coin

70 points: We still want to brute force, but we can do this much much faster.

Insight 1: The first shuffle works on odd and even indices independently.

The hash can be written as h2(h1(x1), h1(x2)) for

h2: u64 -> u64,

h1: u32 -> u32

Only 36/2 = 18 free bits in h1, we can try (almost) all of them.

```python
def hash(x):
    # int to list of bits
    def tolist(x):
        return [(x>>i)&1 for i in range(63, -1, -1)]

    # list of bits to int
    def toint(x):
        return sum([x[63-i]<<i for i in range(0, 64)])

    assert 0 <= x < 2**64
    x = tolist(x)
    for i in range(512):
        y = 4*x[i%64] + 2*x[(i+2)%64] + x[(i+10)%64]
        x[i%64] = [1,1,0,0,1,1,0,1][y]
    x = toint(x)
    for i in range(16):
        x += toint(tolist(x)[::-1])
        x ^= 3**40
        if(x >= 2**64): x >>= 1
    assert 0 <= x < 2**64
    return x
```

Chalmers Coin

Because the hash function is so bad, it turns out the image of h1(x1) and h1(x2) are only a few hundred elements each.

We can loop over all pairs in less than 1000^2 operations. We have now iterated over the entire image of the hash function, and we have found one of the many optimal solutions.

```python
def hash(x):
    # int to list of bits
    def tolist(x):
        return [(x>>i)&1 for i in range(63, -1, -1)]

    # list of bits to int
    def toint(x):
        return sum([x[63-i]<<i for i in range(0, 64)])

    assert 0 <= x < 2**64
    x = tolist(x)
    for i in range(512):
        y = 4*x[i%64] + 2*x[(i+2)%64] + x[(i+10)%64]
        x[i%64] = [1,1,0,0,1,1,0,1][y]
    x = toint(x)
    for i in range(16):
        x += toint(tolist(x)[::-1])
        x ^= 3**40
        if(x >= 2**64): x >>= 1
    assert 0 <= x < 2**64
    return x
```

Chalmers Coin

Why was that only 70 points? Because the hash function is slow. Rewrite h1(u32) -> u32 to work on 16 bits in parallel so that searching the 18 bits is possible in 1 second.

Judge solution takes 300ms. Might be possible in Python.

```
49  u32 comp_xpart(u32 xpart) {
 1      rep(i1, 0, 16) {
 2          u32 a = (xpart >> 16) & 0xFFFF; // 16-31
 3          u32 b = (xpart >> 15) & 0xFFFF; // 15-30
 4          u32 c = (xpart >> 11) & 0xFFFF; // 11-26
 5          u32 val = ((~b) | (a & b & c)) & 0xFFFF;
 6          xpart = (0xFFFF & xpart) | (val << 16);
 7          xpart = (xpart << 16) | (xpart >> 16);
 8      }
 9      return xpart;
10  }
11  u64 comp_hash(u32 xeven, u32 xodd) {
12      u64 x = 0;
13      rep(i, 0, 32) {
14          x |= (u64)((xeven >> i) & 1) << (2*i);
15          x |= (u64)((xodd >> i) & 1) << (2*i + 1);
16      }
17      rep(i, 0, 16) {
18          u64 xflip = __builtin_bswap64(x);
19          xflip = byteswap[xflip & 0xFF]
20              | ((u64)byteswap[(xflip >> 8) & 0xFF] << 8)
21              | ((u64)byteswap[(xflip >> 16) & 0xFF] << 16)
22              | ((u64)byteswap[(xflip >> 24) & 0xFF] << 24)
23              | ((u64)byteswap[(xflip >> 32) & 0xFF] << 32)
24              | ((u64)byteswap[(xflip >> 40) & 0xFF] << 40)
25              | ((u64)byteswap[(xflip >> 48) & 0xFF] << 48)
26              | ((u64)byteswap[(xflip >> 56) & 0xFF] << 56);
27
28          u64 res;
29          bool overflow = __builtin_uaddll_overflow(x, xflip, &res);
30          x = ((res ^ POW3_40) >> overflow) | ((u64)overflow << 63);
31      }
32      return x;
33  }
```

Cookie Game

We had an incredible number of different approaches for different time complexities. The judge solution for 100p is O(n^2 logn), but David Wärn somehow managed without the log factor.